UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

nJavaCalendar

Name: Incze Antonia
Group: 30435/2

Table of Contents

Deliverable 1

Project Specification

nJavaCalendar is a Time Management Application designed to help users stay on top of their schedule with an interactive and easy to use UI.

Functional Requirements

The system shall:

- Allow users to view a weekly schedule displaying time blocks aligned by time
- Allow users to add recurring schedule blocks, representing for example weekly lectures/work
- Allow users to add one-time events for specific dates
- Display time blocks in an alignment by time, where position and size reflect start and end time
- Allow users to edit or delete existing time blocks
- Allow users to assign colors to time blocks
- Allow users to view details of a time block
- Allow users to create and view weekly tasks with no determined time, in the form of a checked list
- Allow users to drag and drop tasks in this list

Use Case Model

Use Cases Identification:

Use-Case: View weekly schedule

Level: User goal

Primary Actor: User

Main success scenario:

- User opens application
- System requests weekly schedule from backend
- System displays 7-day calendar
- Time blocks are shown aligned by time
- User can visually inspect their schedule

Extensions:

- If no events exist, the system displays an empty schedule

Use-Case: Add time block

Level: User goal

Primary Actor: User

Main success scenario:

- User opens “Add Event” form
- User inputs title, day/date, start time, end time and color
- System validates input
- System stores the new block
- System updates calendar view

Extensions:

- Invalid time format results in a validation error

Use-Case: Edit/Delete time block

Level: User goal

Primary Actor: User

Main success scenario:

- User clicks on an existing block
- Edit flow: user modifies details, system validates input and updates block
- Delete flow: user confirms deletion. System removes block
- Calendar refreshes

Extensions:

- Invalid edit results in error
- If a deletion is cancelled, no action is taken

Use-Case: Add Task

Level: User goal

Primary Actor: User

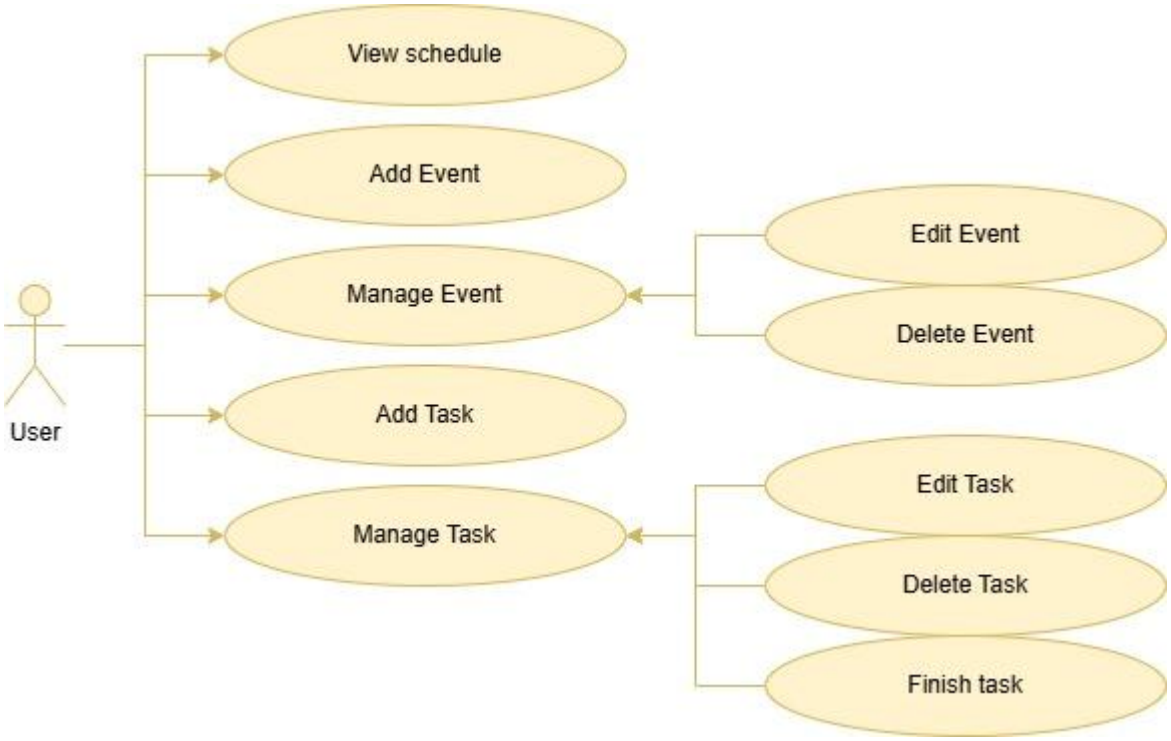
Main success scenario:

- User opens “Add Task” form
- User inputs task description
- System stores task
- Task list refreshes

Extensions:

- Task order may be changed through drag and drop
- Checked tasks disappear at the end of the week
- Possible extension: task history

UML Use Case Diagram



Supplementary Specification

Non-functional Requirements

- The system shall provide an intuitive and visually clear interface for managing schedules
- The system shall load and display the weekly schedule within 1 second under normal conditions
- The system shall prevent inconsistencies such as overlapping time blocks
- The system shall follow a layered architecture

Design Constraints

1. Programming languages

- Backend: Java Spring Boot
- Frontend: React

2. Data Storage

- Currently in-memory, to be extended to database ASAP

3. Development Tools

- IntelliJ IDEA
- Node.js & npm

4. Frontend constraints:

- No external UI frameworks, susceptible to change

Glossary

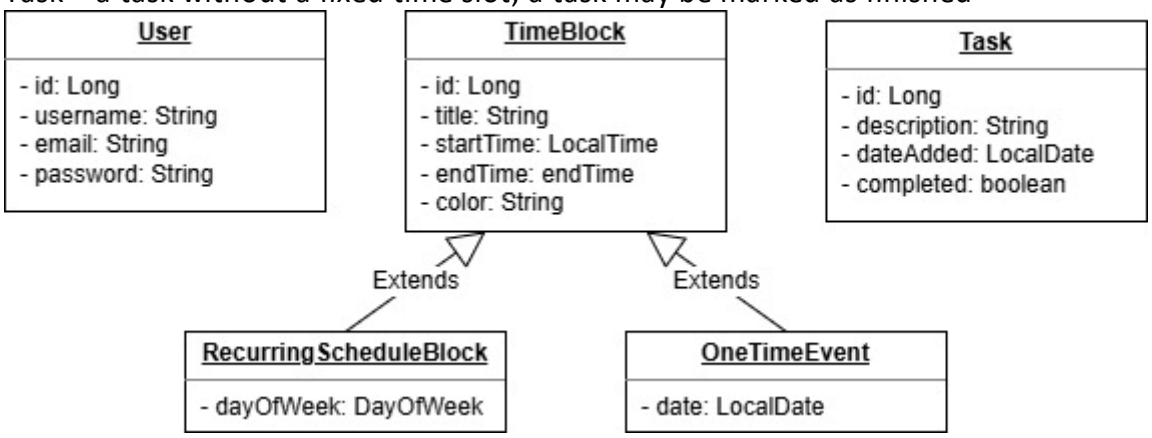
Time Blocks – A scheduled activity with start and end time
Recurring Block – A time block repeated weekly (e.g. lecture)
One-Time Event – A single occurrence event (e.g meeting)
Schedule – A collection of time block across 7 days
Day Schedule – All time blocks for a specific day
DTO – Object used to transfer data between front and backend
View – UI representation of schedule
Edit Mode – UI state allowing modification of blocks

Deliverable 2

Domain Model

The application centers on the idea of a user managing a weekly schedule composed of time blocks and task items. Based on the project specification, the most important domain concepts are recurring schedule blocks, one-time events, and tasks.

User – person using the application; a user owns one schedule and one task collection
Time Block – scheduled activity with a start and end time; it also contains attributes such as title and color
Recurring Time Block – a special time block that represents recurring activities such as school hours or work shifts
One Time Event – a time block that represents a non-recurring event
Task – a task without a fixed time slot; a task may be marked as finished

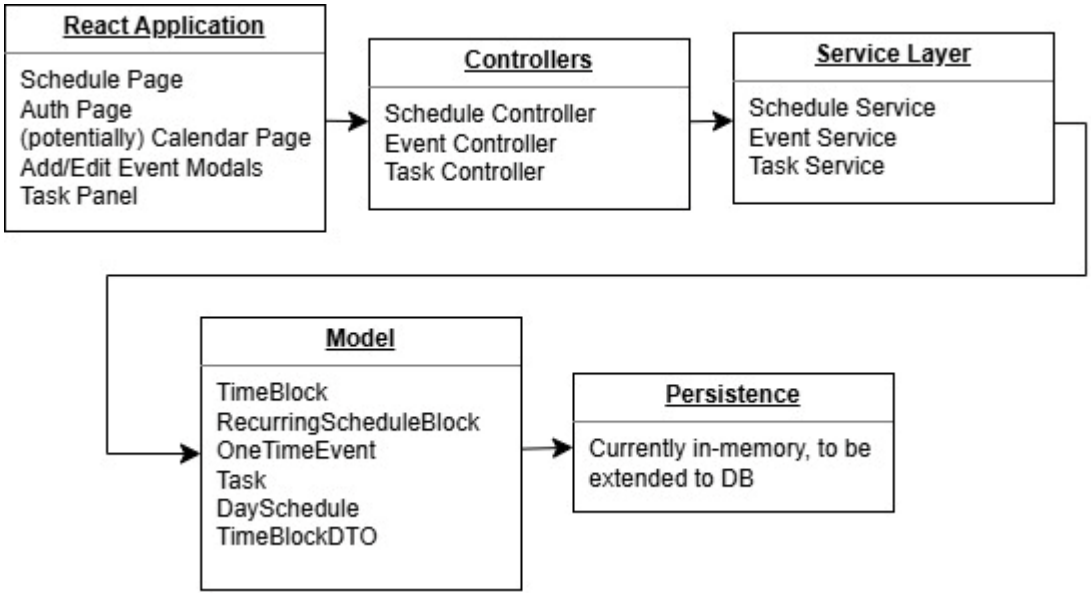


Architectural Design

Conceptual Architecture

Frontend: React application

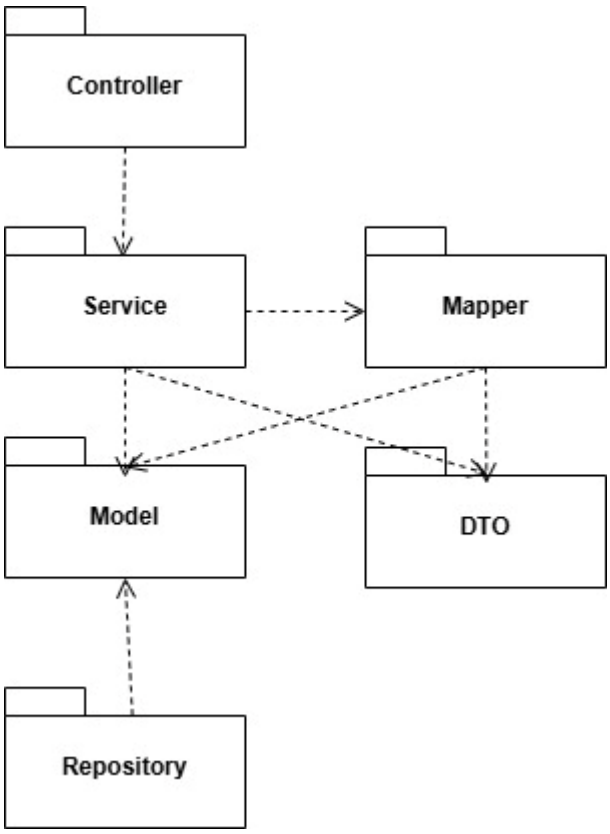
Backend: Spring Boot REST API, Layered Architecture



This is consistent with the project's non-functional requirement to follow a layered architecture.

Package Design

The package design should reflect the layered architecture.



Component and Deployment Diagram

At the current stage, the application can be deployed locally with:

- a browser running the React frontend,
- a Spring Boot server running the backend,

Deliverable 3

Design Model

Dynamic Behavior

[Create the interaction diagrams (2 sequence) for 2 relevant scenarios]

Class Diagram

[Create the UML class diagram; apply GoF patterns and motivate your choice]

Data Model

[Create the data model for the system.]

System Testing

[Describe the testing methides and some test cases.]

Future Improvements

[Present some features that apply to the application scope.]

Conclusion

Bibliography